МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине Построение и анализ алгоритмов

Студент гр. 4343 _____ Пимахин Р.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

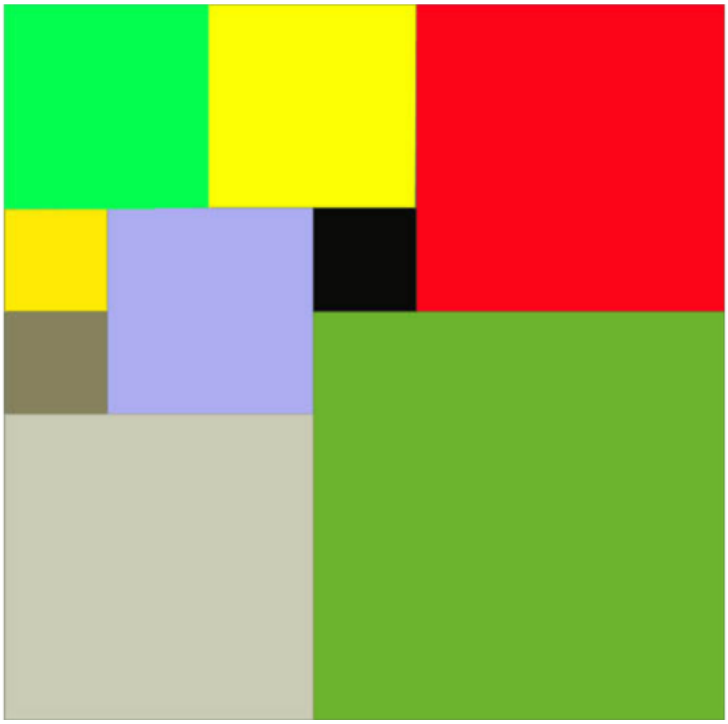
Цель работы

Изучить и реализовать алгоритм поиска с возвратом.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N $(2 \leq N \leq 20)$.

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк,

каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла $(1 \leq x, y \leq N)$ и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 4р.

Рекурсивный бэктрекинг. Расширение задачи на прямоугольные поля, рёбра квадратов меньше рёбер поля. Подсчёт количества вариантов покрытия минимальным числом квадратов.

Описание алгоритма

Для решения задачи используется метод рекурсивного поиска с возвратом (backtracking). Алгоритм начинает работу с пустого поля и последовательно размещает квадраты. На каждом шаге выбирается первая свободная клетка поля. Для нее перебираются возможные размеры квадратов начиная от максимально возможного к минимальному.

Если квадрат можно разместить, он добавляется на поле, после чего выполняется рекурсивный вызов алгоритма для следующего состояния. После завершения поиска в данной ветви выполняется возврат: квадрат удаляется с поля, и происходит переход к следующему варианту размещения.

Основная идея алгоритма заключается в полном переборе возможных покрытий с использованием отсечений, которые позволяют не рассматривать заведомо неэффективные варианты.

Способ хранения частичных решений

Для хранения текущего состояния поля используется двумерный массив grid размером M×N. Каждая ячейка массива соответствует клетке поля:

1) значение 0 означает, что клетка свободна;
2) ненулевое значение означает, что клетка уже занята размещенным квадратом.

При добавлении квадрата функция set_square изменяет значения соответствующих клеток массива, а после завершения рекурсивного вызова выполняется возврат состояния поля обратно путем заполнения этих клеток нулями.

Таким образом, массив grid хранит текущее частичное решение на каждом уровне рекурсии.

Дополнительно используются глобальные переменные: best_square_count - минимальное найденное количество квадратов;

solution_amount - количество найденных вариантов покрытия с минимальным числом квадратов.

Рекурсивная функция

Основная рекурсивная функция:

void backtracking(int squares_count)

Назначение функции: функция выполняет рекурсивный поиск минимального покрытия поля квадратами методом перебора с возвратом.
Аргументы:
squares_count - текущее количество уже размещенных квадратов.

Возвращаемое значение: функция не возвращает значение. Она изменяет глобальные переменные best_square_count и solution_amount, а также изменяет состояние поля grid.
Логика работы функции:
1. Проверяется условие отсечения: если текущее количество квадратов больше уже найденного лучшего решения, то дальнейший поиск в этой ветке прекращается.
2. Выполняется поиск первой свободной клетки поля.
3. Если свободных клеток больше нет, то найдено полное покрытие поля. Проверяется, является ли оно лучшим решением.
4. Для найденной свободной клетки вычисляется максимальный размер возможного квадрата.
5. Перебираются размеры квадратов от максимального к минимальному.
6. При возможности размещения квадрат добавляется на поле, вызывается рекурсивная функция, после чего выполняется возврат состояния поля.

Оценка сложности алгоритма

Временная сложность:

В общем случае временная сложность имеет оценку: O(n^(N^2)), где N - сторона поля. Для прямоугольного поля сложность будет O(n^(M*N)), где M и N – стороны поля.

Сложность по памяти:

Для хранения состояния поля используется массив grid размером M*N: O(M*N).

Также используется стек рекурсивных вызовов. Его глубина зависит от количества квадратов в текущем решении:

O(K), где K - количество квадратов.

Так как размер массива поля обычно больше глубины рекурсии, итоговая сложность памяти: O(M*N).

Оптимизация алгоритма

В алгоритме используются следующие оптимизации:

1. Отсечение по лучшему найденному решению.

   Если текущее количество квадратов уже больше найденного минимального количества, дальнейший перебор этой ветки прекращается.

2. Выбор первой свободной клетки.

   Вместо перебора всех возможных позиций для размещения квадрата выбирается первая незаполненная клетка поля. Это уменьшает количество вариантов.

3. Перебор квадратов от большего размера к меньшему. Сначала рассматриваются большие квадраты, что позволяет быстрее получить хорошие решения и уменьшить количество дальнейших переборов.

4. Проверка возможности размещения перед добавлением квадрата. Перед изменением состояния поля выполняется проверка, что квадрат находится в пределах поля и все его клетки свободны.

Тестирование

Входные данные	Вывод
Квадрат	
4 4	4 1
1 1	Table size must be between 2 and 20

8 8	4 1
9 9	6 4
13 13	11 8
20 20	4 1
Прямоугольник	
3 7	12 32
6 2	12 1
3 5	9 12
12 7	7 2
1 4	Table size must be between 2 and 20
3 4	6 4

Вывод

В ходе лабораторной работы был реализован рекурсивный алгоритм с возвратом, покрывающий квадратное и прямоугольное поля минимальным числом квадратов. Алгоритм успешно находит оптимальное решение для N от 2 до 20 и подсчитывает количество вариантов для прямоугольного поля. Для ускорения поиска были применены оптимизации. Проведенное тестирование показало корректность работы алгоритма.